EXPERIMENT 14

Exploring Pro-Code Extension of a Zoho Creator Application using Zoho Catalyst / Custom Functions

Name: Akshaya S | Register Number: 241401009

Aim

To explore how pro-code capabilities can extend a low-code Zoho Creator application using Zoho Catalyst / custom functions, by creating and executing a custom Deluge function in the Student Admission application.

Introduction

Low-code platforms allow applications to be developed using visual components such as forms, reports and workflows, with little hand-written code. This makes it quick to build standard business applications. When the standard low-code functionality is insufficient for a particular requirement, pro-code extensions allow developers to introduce programmable logic into the application. Zoho Creator supports this through Deluge, its scripting language, and through custom functions, which are reusable units of Deluge code that can be called within an application when required. Zoho Catalyst, Zoho's serverless development platform, represents a further pro-code option for requirements that go beyond what can be scripted inside the application.

In this experiment, a simple custom function named **generateApplicationID** was created for the Student Admission application and executed to observe how a pro-code extension is created and run in Zoho Creator. The function accepts two arguments, applicantName and applicationId, and returns a string.

Pro-Code Extension in Zoho Creator

Extending a low-code application with custom functions means that the visual structure of the application (forms, reports and workflows) is retained, while specific logic is written as code in a function and used where required. The main concepts are as follows:

- **Custom functions:** named, reusable functions created in the Functions section of the Workflow area. A function has a name, a namespace, a return type and optional arguments.
- **Deluge scripting:** the scripting language of Zoho Creator, in which the logic of a custom function is written. The Deluge editor provides a list of ready-made statements such as conditions, data-access operations, set variable, call function, send mail and return.
- **Reusable application logic:** a function is written once and can be called from within the application whenever its logic is needed, which avoids repeating the same code.
- **Custom business logic:** application-specific rules that are not covered by standard configuration can be implemented in a function.
- **Data processing:** Deluge provides operations to add, fetch, aggregate, update and delete records, which allow records to be processed through code.
- **Automation and extensibility:** functions allow automation to be extended beyond standard options, so that the application can be adapted to specific requirements.

Figure 1 illustrates the concept. A user works with the Student Admission application, which has components such as forms, workflows, reports and custom functions. The pro-code extension (Deluge scripting, custom functions, business logic, data processing and automation) is represented by the custom function generateApplicationID(), which accepts the applicant name and application ID, runs Deluge logic and produces a return value.

241401009

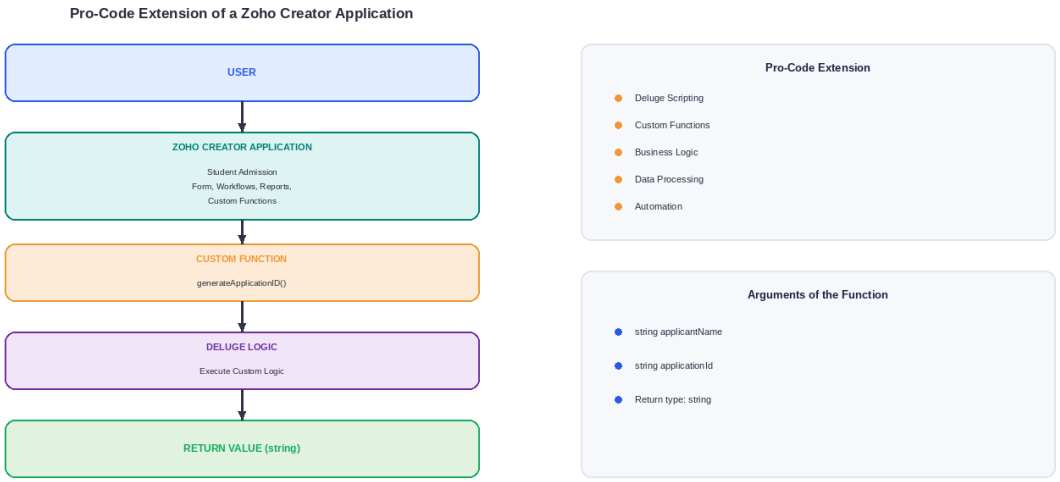


Figure 1: Pro-Code Extension of a Zoho Creator Application (Supporting Diagram)

The diagram is a conceptual illustration. In this experiment, only the creation and execution of the custom function were carried out; the function was not attached to a workflow or connected to any other component.

Procedure

1 Open the Student Admission application in Zoho Creator in Edit mode.

2 Navigate to the Workflow section.

3 Open the Functions section. As no function exists yet, the page shows the message “Execute re-usable functions that can be called within an application when required” with the Create Function option (Figure 2).

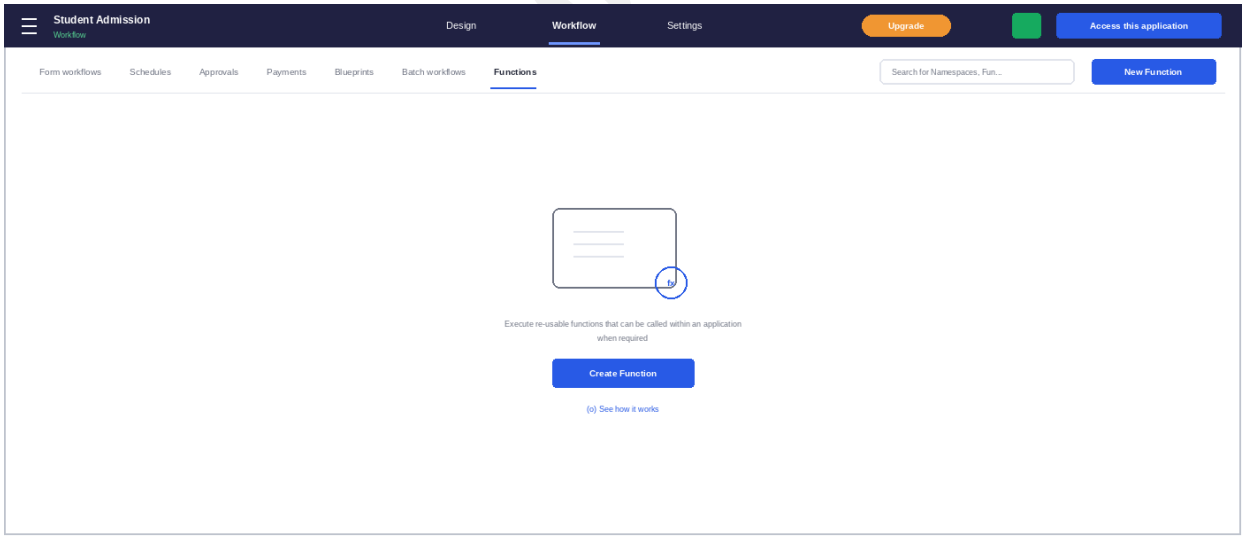


Figure 2: Functions Section of the Workflow Area

4 Select Create Function. The “Execute a function when called” form is displayed, with the fields Name the function, Select the language, Select namespace, Select return type and Specify arguments.

5 Enter the function name generateApplicationID.

6 Select Deluge as the programming language.

7 Use the Default namespace.

8 Set the return type as string.

9 Under Specify arguments, add two arguments of type string: applicantName and applicationId. Then select Create Function (Figure 3).

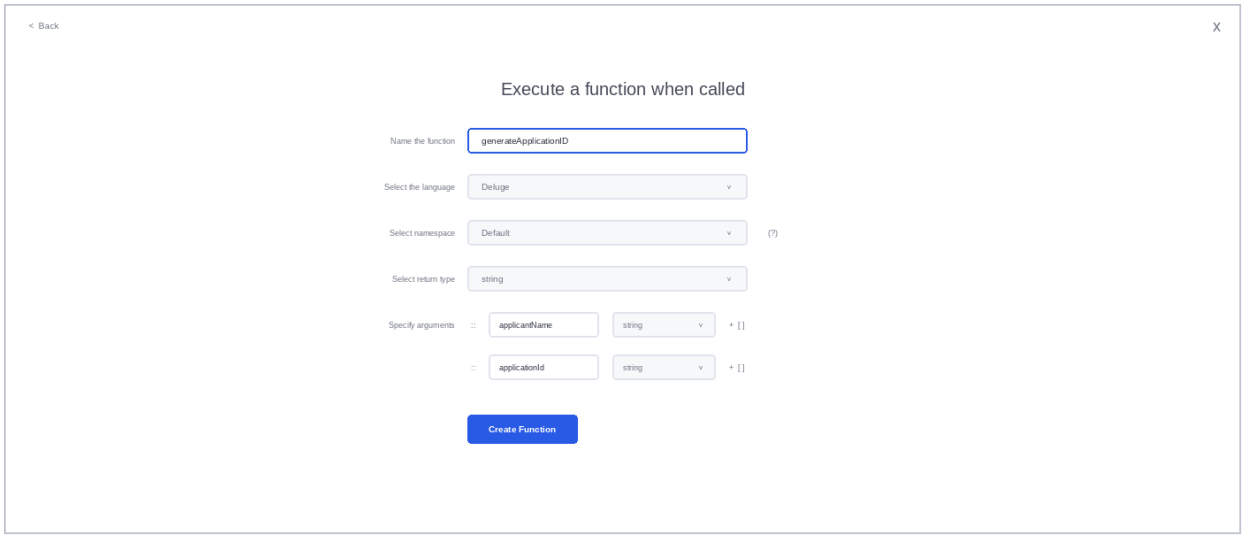


Figure 3: Create Function Form – Name, Language, Namespace, Return Type and Arguments

10 The Deluge editor opens with the function declaration string generateApplicationID(string applicantName, string applicationId) generated from the selected options. Add the Deluge logic to return the confirmation message (shown in the next section), in place of the empty return value generated by default (Figure 4a).

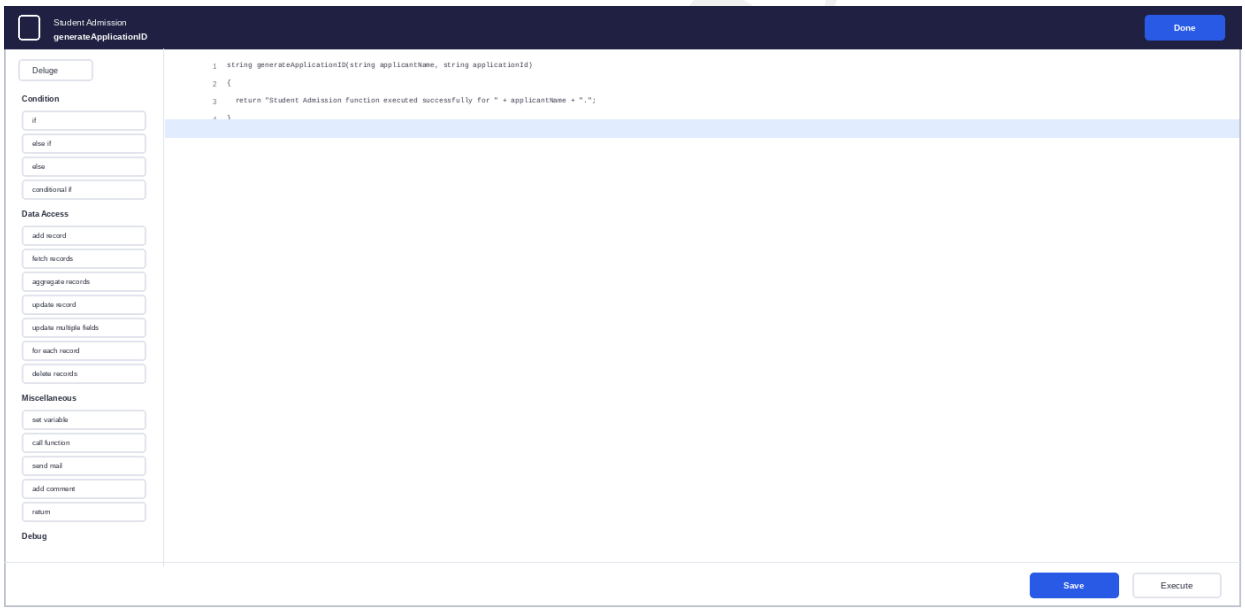


Figure 4(a): Deluge Editor – Function Declaration and Logic for generateApplicationID()

11 Save the custom function using Save.

12 Execute the function using Execute. The Execute window asks for the values of the input arguments; enter John for applicantName and 101 for applicationId, and select Submit (Figure 4b).

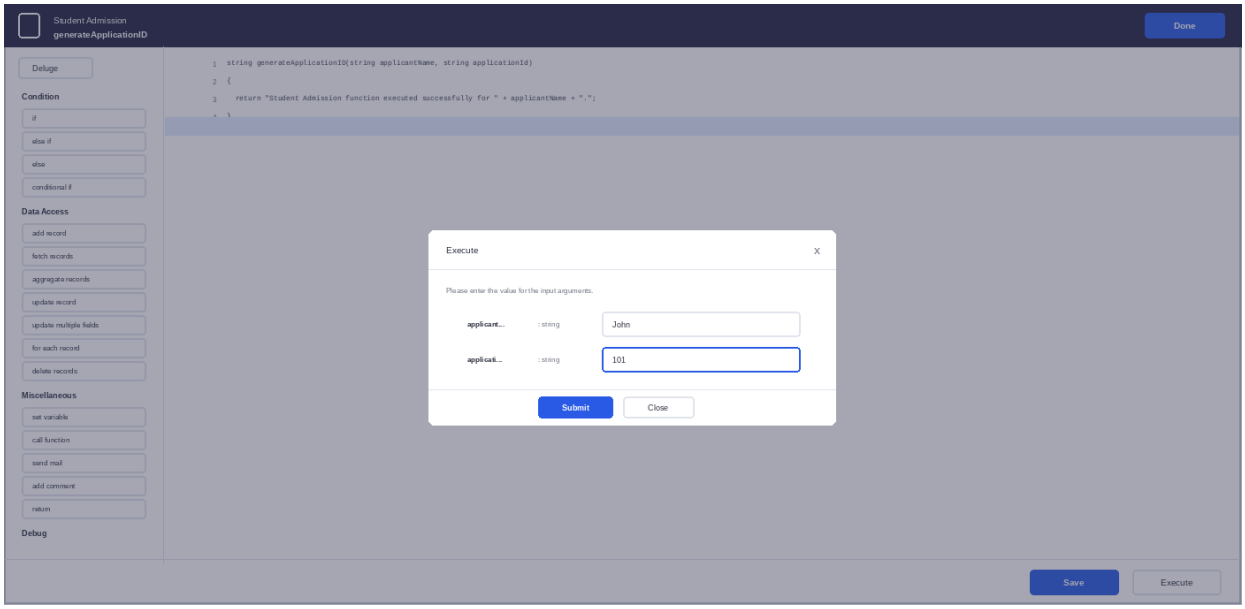


Figure 4(b): Execute Window – Entering the Input Arguments

13 Verify that the Execute window reports Executed Successfully and that the return value is the expected message (Figure 5). The saved function is then listed in the Functions section (Figure 6).

Custom Function Implementation

The following Deluge code was used for the custom function:

```
string generateApplicationID(string applicantName, string applicationId)
{
    return "Student Admission function executed successfully for " + applicantName + ".";
}
```

The function is declared with the return type string and takes two string arguments, applicantName and applicationId. When it is called, it returns the string “Student Admission function executed successfully for ” followed by the applicant name supplied, confirming successful execution. The applicationId argument is declared in the function signature but is not used by the logic in this experiment. The function details are summarised below.

Property	Value
Function Name	generateApplicationID
Language	Deluge
Namespace	Default
Return Type	string
Arguments	string applicantName, string applicationId
Test Input	applicantName = John, applicationId = 101
Return Value	Student Admission function executed successfully for John.

Function Execution

The custom function was executed in Zoho Creator using the Execute option of the Deluge editor, with the values John and 101 supplied as arguments. The Execute window displayed the result Executed Successfully and the return value “Student Admission function executed successfully for John”, which is the expected confirmation message. The code shown behind the window is the saved function, with the message in the return statement.

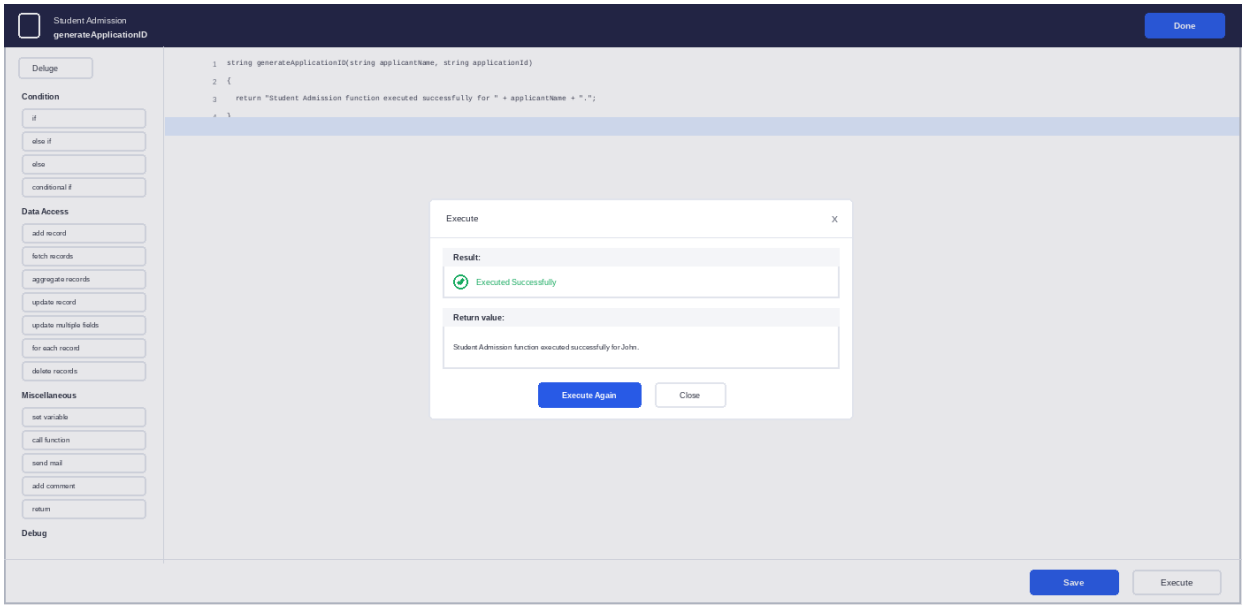


Figure 5: Execution of generateApplicationID() Custom Function

Saved Custom Function

After saving, the function appears in the Functions section of the Workflow area of the Student Admission application. It is listed under the Default namespace with the function name generateApplicationID, the language Deluge, the return type string and the syntax generateApplicationID(string applicantName, string applicationId).

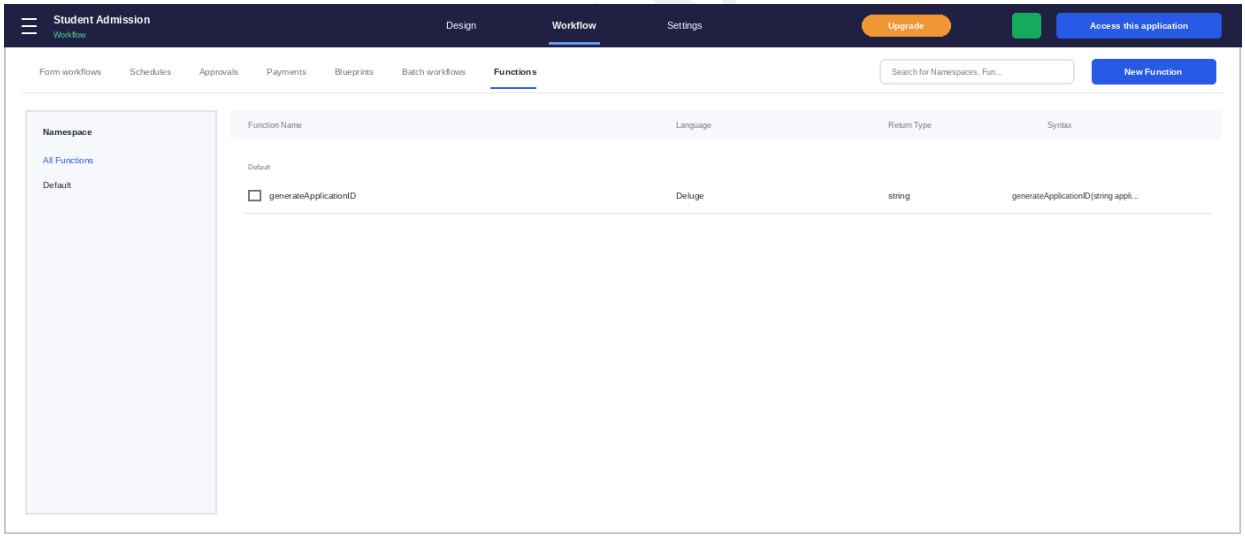


Figure 6: Saved generateApplicationID Custom Function

Execution Flow

Figure 7 shows the execution flow of the custom function. The flow is: Student Admission Application → generateApplicationID() → Deluge Function Execution → Custom Logic → String Return Value → Successful Execution.

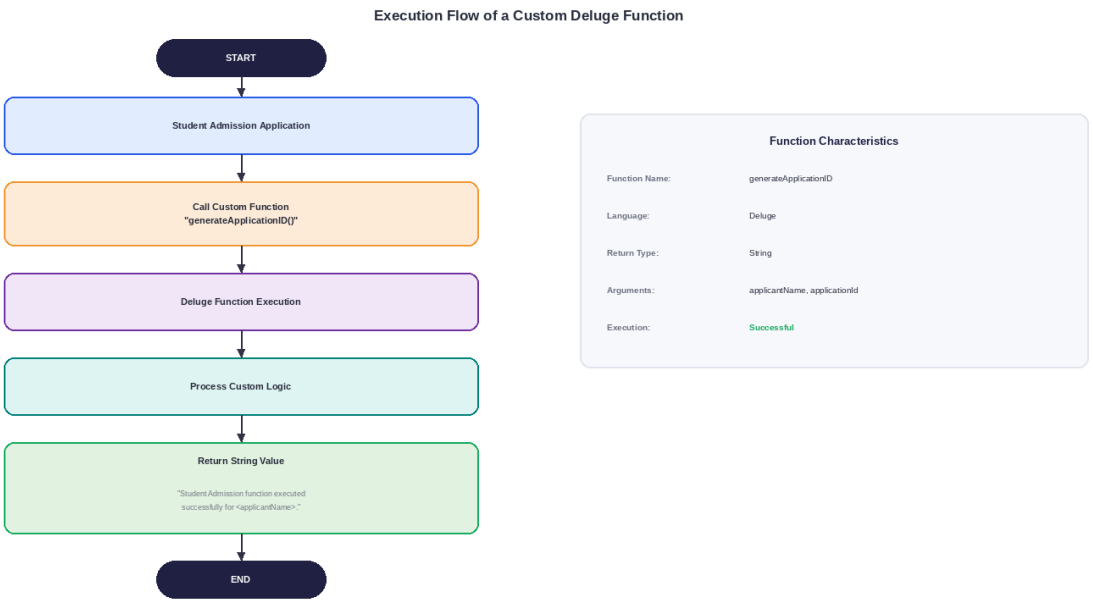


Figure 7: Execution Flow of a Custom Deluge Function (Supporting Diagram)

- The application calls the custom function generateApplicationID() with the arguments applicantName and applicationId.
- The Deluge function is executed and its custom logic is processed. In this experiment the logic consists of returning a message that includes the applicant name.
- A string value is returned, and the function completes with successful execution, as confirmed in Figure 5.

Low-Code and Pro-Code Extension

Low-code components provide the basic structure of an application, while pro-code extensions allow additional programmable logic and customisation. Figure 8 presents the two together as a combined approach.

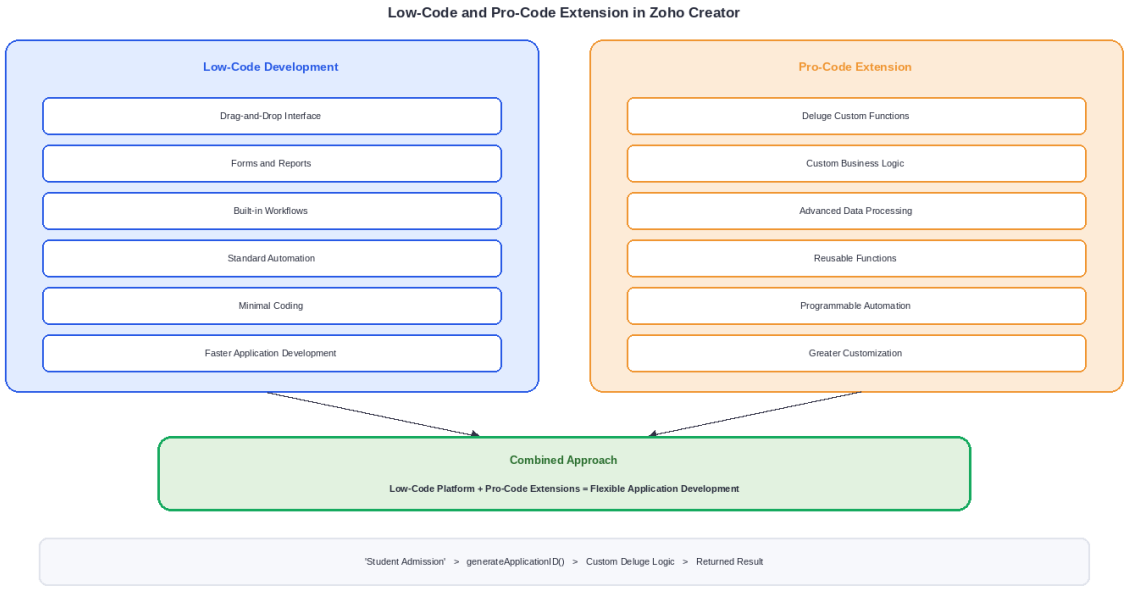


Figure 8: Low-Code and Pro-Code Extension in Zoho Creator (Supporting Diagram)

Low-Code	Pro-Code
Forms	Deluge functions
Reports	Custom logic
Workflows	Advanced processing
Standard automation	Reusable functions
Visual development	Programmable automation

The comparison describes the general roles of the two approaches. This experiment demonstrated only a simple custom function that returns a confirmation message; it did not implement application ID generation, advanced processing or integration with workflows or other systems.

Result

The custom Deluge function generateApplicationID was successfully created in the Student Admission application using the Default namespace, the string return type and the arguments applicantName and applicationId. The function was saved, listed in the Functions section and executed with the test values John and 101, returning the expected confirmation message.

Conclusion

Low-code platforms such as Zoho Creator simplify application development by providing forms, reports and workflows that can be assembled visually. Where specific requirements go beyond the standard components, pro-code extensions provide additional flexibility, and custom Deluge functions can introduce application-specific logic in a reusable form.

In this experiment, a custom function named generateApplicationID was created with the Deluge language, the Default namespace, the string return type and two string arguments in the Student Admission application. The function was saved, listed in the Functions section and executed successfully, returning the expected confirmation message for the supplied applicant name. Although the function was simple, it illustrates the procedure for creating, executing and verifying a custom function. Combining low-code development with suitable pro-code extensions can support more flexible application development, in which standard functionality is built visually and specific logic is added through code where it is required.

241401009